

Simply a Basic Variant

- Document number: P0094R0
- Date: 2015-09-24
- Project: ISO/IEC JTC1 SC22 WG21
- Reply-to: David Sankel david@stellarscience.com

Introduction

There is wide agreement that C++ needs a type-safe union type and the significant number of "variant" implementations in the wild confirms that it fills a practical need for many developers.¹ On the other hand, how exactly to implement a variant has been an item of much controversy of late. This is witnessed by many hundreds of variant emails on the standard library mailing list and the heated discussions invoked by related blog posts.²³ The danger, of course, with all the controversy is that consensus will not be achieved and that software developers will suffer the effects of widespread divergent implementations of a library considered to be an essential "vocabulary" type.

This paper is an attempt to bridge the divide by proposing a hitherto undiscussed option. To the author's surprise, this attempt at a compromise has the unusual properties of being both more flexible and more simple (in both specification and usage) than the competing alternatives.

In a nutshell, we are proposing a variant implementation that has the never-empty guarantee, preserves **basic** type safety, and gives implementers the flexibility to optimize their implementations based on the alternative types. As it turns out, this simple specification meets the needs of opposing camps and provides an interface that will allow more optimizations as language improvements come about in the future.

Design Considerations

Never empty

At the May 2015 meeting at Lenexa there was consensus that the variant type should model the mathematical concept of the discriminated union. This decision led to more consistency and provided guidance as to what the interface should look like. This is particularly true with regard to whether or not to allow repeated alternative types and how to handle conversions.

Although this consensus was achieved at Lenexa, there was a lingering contradiction between the attempt to model a discriminated union and the proposed implementation; namely, there is no mathematical equivalent of an "invalid" state. Although the committee took pains to limit the conditions required to get into the invalid state, it nonetheless complicated the semantics of variant. Consider the following function declaration:

```
void f( std::variant<A,B,C> & );
```

f may or may not be able to properly handle an "invalid" variant. Although most functions with variant parameters do not *need* to handle an "invalid" variant, there are some that do and we need to distinguish between them. This can be somewhat obviated by having a coding standard that states "unless otherwise specified, variant parameters have a validity precondition", but the extra complication of the semantics still remains.

The variant of this proposal completely removes the "invalid" state and, thus, has no difficulties modeling a discriminated union.

Why Basic?

Although a variant with only basic exception guarantees is difficult to use in a context where a strong guarantee is required ⁴, double buffering can only be elided when all alternative types meet certain criteria. This is in contrast to a variant with basic guarantees where only one alternative type needs to meet certain criteria to elide double buffering. This is discussed in more detail in the following section.

Performance implications

A naive implementation of the proposed library would use double buffering as a means to provide basic exception safety guarantees. This implementation path has an important drawback: every variant instance would have a size that is twice that of the largest alternative type. None of the other variants proposed thus far have this drawback.

On closer inspection, however, one can see that not all variants would require double buffering. For example, if every

alternative of a variant type has a non-throwing move constructor there is no need for double buffering. This is notably true for all built-in types. A non-double buffered variant could also be created if alternatives only supported non-throwing move assignment operations by using the stack for temporary values. This applies, or could apply, to most standard types.

All of the above scenarios also apply to a variant with strong exception safety guarantees.⁵ The variant with basic exception safety guarantees, on the other hand, has one other optimization scenario: If one of the alternatives has a no-throw default constructor, there is no need for double buffering. In this case, when an exception is thrown during assignment the implementation has the option to set the value of the variant to the default-constructed value of the type with the no-throw default constructor.

While this proposal does not require implementations to take advantage of these optimizations, a quality implementation would make use of these strategies when possible, relieving the developer from these concerns.

One oft-cited concern is developers who are writing high-performance code. The underlying assumption is that they would not find double-buffering acceptable under any circumstances. These developers could easily add a no-throw default constructable alternative to their variant use case (perhaps using `std::monotype`⁶) and have confidence that double-buffering is disabled.

Conclusion

The need for a standard variant type is clear. This paper proposes a variant that is simply defined, is easy to use, and meets a wide variety of needs.

-
1. Variant: a type-safe union (v4). N4542↩
 2. [Standardizing Variant: Difficult Decisions](#)↩
 3. [A variant for the everyday Joe](#)↩
 4. Simply a Strong Variant. P0093↩
 5. Simply a Strong Variant. P0093↩
 6. Variant: a type-safe union (v4). N4542↩